UNIVERSIDAD TECNOLÓGICA DE SAN JUAN DEL RÍO

ACTIVIDAD

ALGORITMOS BÁSICOS DE ORDENAMIENTO

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL / DESARROLLO DE SOFTWARE MULTIPLATAFORMA / JALPAN

PRESENTA:

ESPINOZA HERNANDEZ CELESTE

PROFESOR DE ASIGNATURA

MARÍA DE MONSERRATH GUERRERO CHAVERO

JALPAN DE SERRA, QRO.

8 DE OCTUBRE DE 2026

INTRODUCCIÓN

Los algoritmos de ordenamiento son fundamentales en la programación y el manejo de estructuras de datos, ya que permiten organizar un conjunto de elementos de acuerdo con un criterio específico, como el orden ascendente o descendente. Su aplicación facilita la búsqueda, la consulta y el procesamiento de la información, contribuyendo a que los programas sean más eficientes y fáciles de utilizar. Entre los algoritmos básicos de ordenamiento se encuentran el método de burbuja, el método de inserción y el método de selección, los cuales utilizan diferentes procedimientos para comparar y acomodar los elementos de una colección.

El estudio de estos algoritmos permite comprender cómo se modifica la posición de los datos durante el proceso de ordenamiento, así como identificar las diferencias entre sus procedimientos y su rendimiento. Mediante la simulación paso a paso de las comparaciones, los desplazamientos y los intercambios en memoria, es posible observar de manera visual cómo cada algoritmo transforma una lista desordenada en una lista ordenada. Asimismo, el análisis de su complejidad ayuda a conocer la cantidad de operaciones que pueden realizar y a comprender la importancia de seleccionar un método adecuado según las características y el tamaño de los datos que se desean procesar.

Algoritmos básicos

Algoritmos de ordenamiento

Ordenación de burbuja

El algoritmo de ordenación de burbuja es el mas sencillo y funciona intercambiando repetidamente los elementos adyacentes si están en el orden incorrecto. Este algoritmo es eficiente para grandes conjuntos de datos, ya que su complejidad temporal, tanto en el caso promedio como en el peor, es bastante alta.

- Ordena el array mediante varias pasadas. Tras la primera pasada, el elemento máximo se coloca al final (en su posición correcta). De igual forma, tras la segunda pasada el segundo elemento más grande se coloca en la penúltima posición, y así sucesivamente.
- En cada pasada, procesa solo aquellos que aún no se hayan movido a la posición correcta. Después de K pasada, el mayor valor de K debe haberse movido a las ultimas K posiciones.
- En cada pasada, consideramos los elementos restantes, comparamos todos los adyacentes e intercambiamos si un elemento mayor precede a uno menor

Python

```
#Programa optimizado para la implementación del
algoritmo de ordenación de burbuja

def bubbleSort (arr):
    n = len (arr)

    #Recorrer todos los elementos del array
    for i in range (n):
        swapped = False

        #Los últimos elementos i ya están en su lugar
        for j in range (0, n-i-1):

            #Recorre el array desde 0 hasta n-1
            #Intercambiar si el elemento encontrado es mayor
            #que el siguiente elemento
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if (swapped == False):
            break

#Codigo del controlador para probar lo anterior
if __name__ == "__main__":
    arr =[64, 34, 25, 12, 22, 11, 90]
```

```
bubbleSort(arr)

    print("Sorted array:")
    for i in range(len(arr)):
        print("%d" % arr[i], end=" ")
```

Producción

Matriz ordenada

11 12 22 25 34 64 90

Análisis de complejidad:

- Complejidad temporal: $O(n^2)$
- Espacio auxiliar: $O(1)$

Ventajas

- El algoritmo es facil de entender
- No requiere espacio de memoria adicional
- Se trata de un algoritmo de ordenamiento estable

Desventajas

- Complejidad temporal de $O(n^2)$ lo que lo hace muy lento para conjuntos de datos grandes
- No tiene aplicaciones practicas. Se usa principalmente en el ambito academico

Ordenacion por inserción

El ordenamiento por inserción, a un que sigue siendo $O(n^2)$, funciona de una manera ligeramente diferente. Siempre mantiene una sublista ordenada en las posiciones inferiores de la lista. Cada ítem nuevo se "inserta" de vuelta en la sublista previa de manera que la sublista ordenada sea un item más larga

Comenzamos asumiendo que una lista con un item (posición 0) ya esta ordenada. En cada pasada, una para cada item desde 1 hasta $n-1$, el item actual se comparara contra los de la sublista ya ordenada. A medida que revisamos en la sublista ya ordenada, desplazamos a la derecha los ítems que sean mayores. Cuando llegamos a un item menor o al final de la sublista, se puede incertar el item actual

Java

```
public class OrdenacionInsercion {
    public static void main (String[] args) {
        int[] numeros = {5, 3, 8, 4, 2};

        System.out.println("Arreglo inicial:");
        mostrar(numeros);

        // Recorremos desde el segundo elemento
        for(int i = 1; i < numeros.length; i++){
            int clave = numeros[i]
            int j = i - 1;

            // Recorremos los elementos mayores a la derecha
            while (j >= 0 && numeros[j] > clave) {
                numeros[j + 1] = numeros[j];
                j--;
            }

            // Insertamos el elemento en su posición correcta
            numeros[j + 1] = clave;

            System.out.println("Pasada" + i + ":");
            mostrar(numeros);
        }
    }
}
```

```
public static void mostrar(int[] numeros){
    for(int numero : numeros){
        System.out.println(numero + " ");
    }
    System.out.println();
  }
}
```

Resultado

Arreglo inicial:

5 3 8 4 2

Pasada 1:

3 5 8 4 2

Pasada 2:

3 5 8 4 2

Pasada 3:

3 4 5 8 2

Pasada 4:

2 3 4 5 8

SECRETARÍA DE EDUCACIÓN · QUERÉTARO GOBIERNO DEL ESTADO Juntos, Adelante. · CONTIGO TODOS AVANZAMOS

Ordenación por QuickSort

Es un algoritmo que utiliza la estrategia divide y vencerás. Selecciona un elemento llamado pivote, divide los elementos en grupos menores y mayores que él y ordena recursivamente cada grupo.

Ejemplo: ordenar [4, 2, 5, 1, 3]

Arreglo inicial

4 2 5 1 3

Se elige 3 como pivote para ilustrar el proceso

↓

Menores	Pivote	Mayores
[2, 1]	[3]	[4, 5]

↓

Resultado ordenado

1 2 3 4 5

Complejidad

Mejor y promedio $O(n \log n)$; peor caso $O(n^2)$, por ejemplo, si se eligen pivotes que producen peticiones muy desequilibradas repetidamente. Su espacio auxiliar suele ser $O(\log n)$ en promedio por la recursión, aunque puede llegar a $O(n)$ en el peor caso.

Algoritmos de búsqueda

La búsque permite localizar un libro por su identificador, título u otro dato. Los dos métodos básicos son la búsqueda lineal y la búsqueda binaria.

Busqueda lineal o secuencial

Recorre los elementos uno por uno hasta encontrar el valor buscado o llegar al final.

Ejemplo:

Buscar el libro con ID 30 en [10, 20, 30, 40, 50].
Primero revisa el 10, despues el 20 y finalmente el 30
En la tercera comparación encuentra el resultado.

- Mejor caso: $O(1)$, si el elemento está al inicio
- Caso promedio y peor: $O(n)$.
- Funciona con datos ordenados o desordenados

Busqueda binaria

Divide repetidamente el rango de búsqueda a la mitad. Para utilizarla, los datos deben estar ordenados.

Ejemplo:

Buscar el ID 40 en [10, 20, 30, 40, 50, 60, 70]. Se revisa el elemento central, 40 y se encuentra inmediatamente.

- Mejor caso: O(1)
- Caso promedio y peor: O(log n)
- Requiere datos ordenados y acceso eficiente al elemento central.

Big-O

La notación Big-O describe como crece el trabajo o el uso de memoria de un algoritmo conforme aumenta la cantidad de datos. No indica un tiempo exacto en segundos, sino una tasa de crecimiento.

O(1) Constante: El trabajo no aumenta con la cantidad de datos.

Ejemplo: Consultar el primer elemento de un arreglo

O(log n) Logaritmica: El espacio de busqueda se se reduce por etapas

Ejemplo: busqueda binaria en un arreglo ordenado

O(n) Lineal: El trabajo crece proporcionalmente a los datos

Ejemplo: recorer una lista enlazada

SECRETARÍA DE EDUCACIÓN · QUERÉTARO GOBIERNO DEL ESTADO Juntos, Adelante. · CONTIGO TODOS AVANZAMOS

$O(n \log n)$ Lineal logaritmica: Crecimiento habitual de algoritmos eficientes de ordenamiento

Ejemplo: QuickSort en el caso promedio

$O(n^2)$ Cuadratica: El trabajo puede crecer como el cuadrado del tamaño de entrada

Ejemplo: Burbuja e inserción en el peor caso

CONCLUSIÓN

El análisis de los algoritmos de ordenamiento de burbuja, inserción y selección permite comprender que existen diferentes formas de organizar los datos, aunque todos estos métodos tienen el mismo objetivo: obtener una secuencia ordenada mediante comparaciones y modificaciones en la posición de sus elementos. A través de la simulación paso a paso, se puede identificar cómo trabaja cada algoritmo, qué operaciones realiza y de qué manera cambia el contenido de la memoria hasta alcanzar el resultado esperado.

Además, conocer la complejidad de estos algoritmos es importante para evaluar su eficiencia y reconocer sus ventajas y limitaciones. Aunque los tres métodos son útiles para aprender los fundamentos del ordenamiento, su rendimiento puede variar dependiendo de la cantidad y la distribución de los datos. En conclusión, el estudio de estos procedimientos fortalece la comprensión de las estructuras de datos, desarrolla el razonamiento lógico y proporciona una base para abordar algoritmos más avanzados, como QuickSort, que permiten trabajar con conjuntos de información de mayor tamaño.